Marwadi University
Faculty of Engineering and Technology
Department of Information and Communication Technology

Veterinary Management System

Created by: 92400133037- Heer Mehta

92400133169- Rishi Sampat

Submitted to: Prof. Drashti

Faculty of DBMS/ICT

INDEX

SR. NO.	Topics	Page No.
1	Abstract	3
2	Objectives	4
3	System Modules	5
4	ER Diagram	9
5	DataBase Schema	10
6	Normalization	13
7	Queries (LAB PART)/ Procedures/ Triggers	15
8	Snapshots	16
9	Future Scope	25
10	Conclusion	26

ABSTRACT

The Veterinary Management System is a comprehensive, database-driven application developed to streamline and integrate the various operations of a modern veterinary clinic. It brings together clinical management, e-commerce services, pet care facilities, and inventory tracking into a single unified platform. The main objective of the system is to efficiently handle patient records, appointments, medical prescriptions, and laboratory reports, while also enabling online product purchases and grooming service bookings.

The system is built using a relational database model implemented in MySQL, ensuring data integrity, consistency, and scalability. Key entities such as users, pets, doctors, products, orders, and services are organized into well-structured, normalized tables to reduce redundancy and maintain efficient data management. Relationships between these entities are defined using primary and foreign keys, closely reflecting real-world interactions within the system.

To enhance performance and maintain data accuracy, advanced database concepts such as normalization (up to Third Normal Form), indexing, and constraints are applied. Stored procedures are used to handle commonly performed operations like appointment booking and order management, while triggers automate essential tasks such as updating inventory and checking stock availability. Views are implemented to simplify complex queries and ensure secure access to data.

The system also incorporates role-based access control, allowing different types of users—including administrators, doctors, receptionists, lab technicians, beauticians, and customers to interact with the system according to their specific roles and responsibilities. This approach improves both usability and security.

Overall, this project demonstrates a practical, real-world application of database management concepts by integrating healthcare, retail, and service modules into a single digital ecosystem. It highlights the importance of efficient database design, automation, and data integrity in developing scalable and reliable systems.

OBJECTIVES

The primary objective of the Veterinary Management System is to design and implement a robust and efficient database system that brings together different aspects of veterinary clinic operations into one unified platform. The system focuses on improving data organization, accessibility, and overall operational efficiency, while ensuring data integrity and scalability.

The key objectives of the project are as follows:

- To manage patient and pet records efficiently
- The system maintains detailed information about pets, including their medical history, appointments, and treatments, ensuring quick access and improved healthcare management.
- To streamline appointment scheduling and clinical operations
- It supports the booking, tracking, and management of appointments between pet owners and doctors, helping reduce manual errors and enhance service efficiency.
- To handle medical prescriptions and laboratory reports
- The system stores prescriptions and test results in a well-structured format, making it easier for doctors and staff to access and update medical information when needed.
- To integrate an e-commerce module for pet-related products
- It allows users to browse, purchase, and track orders for medicines, pet food, toys, and grooming products through an online interface.
- To maintain inventory and stock management
- The system monitors product availability, automatically updates stock levels, and records inventory changes to avoid inconsistencies and shortages.
- To provide pet grooming and care service management
- It enables the booking and allocation of grooming services such as bathing, cleaning, and hair cutting, improving convenience for customers.
- To implement role-based access control
- Different users including administrators, doctors, receptionists, lab technicians, beauticians, and customers are given controlled access based on their roles and responsibilities.
- To ensure data integrity and minimize redundancy through normalization
- The database is designed using normalization techniques (up to Third Normal Form) to maintain consistency and eliminate duplication.
- To automate operations using advanced DBMS features
- Stored procedures, triggers, and constraints are utilized to automate tasks such as stock updates, validation, and reporting.
- To design a scalable and real-world applicable database system
- The system is structured to support future enhancements like online payments, analytics, and mobile integration.

SYSTEM MODULES

1. Clinic Management Module

This module manages all medical and administrative activities related to veterinary healthcare. It maintains detailed pet records, including medical history, treatments, and appointments. Doctors can create prescriptions, and laboratory reports are linked with appointments to maintain a complete medical profile.

From a database perspective, relationships are carefully designed to avoid redundancy and maintain consistency. A prescription can contain multiple medicines, and a medicine can be part of multiple prescriptions, forming a many-to-many relationship.

This is resolved using a junction table.

Tables involved:

Prescription

Product (Medicines)

PrescriptionItem (PrescriptionID, ProductID, dosage, quantity)

This structure ensures that medicine details are stored only once while allowing multiple associations, thereby eliminating duplication and maintaining normalization.

2. E-Commerce Module

The E-Commerce Module enables users to browse and purchase pet-related products such as medicines, food, toys, and grooming items. It supports cart management, order placement, delivery tracking, and payment handling.

From a database perspective:

A user can add multiple products to a cart

A product can appear in multiple carts

This creates a many-to-many relationship, resolved using an intermediate table.

Tables involved:

Users

Cart

CartItem (CartID, ProductID, quantity)

Product

Each user is associated with one cart, and each cart can contain multiple items. Product details are stored separately to avoid redundancy and prevent anomalies such as duplication and deletion issues.

3. Order and Transaction Management

Once products are purchased, the system manages orders, payments, and deliveries. A user can place multiple orders, and each order can contain multiple products.

Relationships:

User → Orders (1:N)

Order → OrderItems (1:N)

Product → OrderItems (1:N across orders)

A composite key (OrderID, ProductID) is used in the OrderItem table to ensure uniqueness and prevent duplicate entries.

Normalization Insight:

Instead of storing product details in OrderItems:

(orderID, productID, productName, price) → causes redundancy

Product details stored separately in Products table

Final Tables:

Products (productID, name, price)

OrderItems (orderID, productID, quantity)

Real-world considerations:

An order can have multiple payment records (not strictly 1:1)

Delivery is handled separately for flexibility

Product can appear in multiple orders

4. Grooming and Pet Care Module

This module manages grooming services such as bathing, cleaning, nail trimming, and hair cutting. Users can book services by selecting time slots and pets.

Tables involved:

Services

Service_Bookings

Beauticians

Constraints:

Scheduling conflicts are avoided

Multiple appointments cannot be booked at the same time for the same resource

This ensures efficient service allocation and realistic system behavior.

5. Inventory Management Module

The Inventory Module tracks product stock used in both clinic and e-commerce operations. It maintains real-time stock levels and automatically updates inventory when products are sold or used in prescriptions.

Tables involved:

Inventory

Stock_Log

Key Design Points:

Product data is stored separately from transactions

Prevents update, insert, and delete anomalies

Maintains accurate logs of stock changes

6. User and Role Management

The system supports multiple user roles such as administrators, doctors, receptionists, lab technicians, beauticians, and customers.

Case 1: Simple Design

1 Role → Many Users (1:N)

Case 2: Real-world Scenario

A user can have multiple roles (e.g., manager + receptionist)

→ Leads to Many-to-Many relationship

Solution:

Tables:

Users (userID, name, roleID) OR base info

Roles (roleID, roleName)

User_Role (userID, roleID)

Foreign key constraints enforce referential integrity, and cascading ensures consistency during updates and deletions.

7. Address and Real-World Constraints

To reflect real-world usage:

A user can have multiple addresses (home, office, etc.)

Orders are linked to selected addresses

This improves flexibility and usability.

8. Normalization and Data Integrity

The database design follows normalization principles:

Removal of partial dependencies

Separation of entities based on functional dependency

Avoidance of redundant data storage

Examples:

productID → product details

roleID → roleName

This ensures:

Efficient storage

Better query performance

Reduced anomalies

ER-DIAGRAM

https://drive.google.com/drive/folders/1Fe1j_fwxDPYLVFAhfo4h-KsYNxlzrn1f?usp=sharing

DATABASE SCHEMA

User & Role Management

1) **Roles** *(roleID, roleName)*
   Stores different roles such as Admin, Doctor, User, Beautician, and Lab Technician.
2) **Users** *(userID, userName, email, password)*
   Stores basic user information.
3) **User_Phones** *(phoneID, userID, phone)*
   Stores multiple phone numbers for a user (multivalued attribute handling).
4) **User_Roles** *(userID, roleID)*
   Junction table implementing many-to-many relationship between users and roles.

Pet & Clinic Management

5) **Pets** *(petID, userID, petName, species, breed, age, gender)*
   Stores pet details linked to their owners.
6) **Doctors** *(doctorID, userID, specialization)*
   Stores doctor-specific details (specialization).
7) **Appointments** *(appointmentID, petID, doctorID, appointmentDate, appointmentStatus)*
   Manages scheduling between pets and doctors.

Products & Categories

8) **Categories** *(catID, catName)*
   Stores product categories (Food, Toys, Medicine, etc.).
9) **Products** *(productID, proName, catID, price, description)*
   Stores all product-related details.

Prescription Management

10) **Prescriptions** *(prescriptionID, appointmentID, notes)*
    Stores prescriptions given during appointments.
11) **PrescriptionItems** *(itemID, prescriptionID, productID, quantity)*
    Stores medicines in each prescription (resolves many-to-many relationship between prescriptions and products).

Shopping Cart & Orders

12) **Cart** *(cartID, userID)*
Stores user cart.

13) **Cart_Items** *(cartItemID, cartID, productID, quantity)*
Stores products added to cart.

14) **Addresses** *(addressID, userID, addressLine, city, state, country, pincode)*
Stores user delivery addresses.

15) **Orders** *(orderID, userID, orderDate, totalAmt, orderStatus, addressID)*
Stores order details.

16) **OrderItems** *(orderItemID, orderID, productID, quantity, price)*
Stores products in each order (junction table).

17) **Payments** *(paymentID, orderID, amt, method, payStatus)*
Stores payment details.

18) **Deliveries** *(deliveryID, orderID, deliveryStatus, deliveryDate)*
Tracks delivery status.

Grooming & Services

19) **Services** *(serviceID, serviceName, price, durationMins)*
Stores grooming services.

20) **Beauticians** *(beauticianID, userID)*
Stores beautician details.

21) **BeauticianSchedule** *(scheduleID, beauticianID, availDay, startTime, endTime)*
Stores beautician availability.

22) **ServiceBookings** *(bookingID, userID, petID, serviceID, bookingDate, bookingTime, serviceStatus)*
Stores service bookings.

23) **ServiceAssign** *(assignID, bookingID, beauticianID)*
Assigns beauticians to bookings (junction table).

Inventory Management

24) **Inventory** *(invID, productID, quantity, expiryDate)*
Stores current stock.

25) **StockLog** *(logID, productID, changeType, quantity, logDate)*
Maintains stock history.

Laboratory Management

26) **Tests** *(testID, testName, testDescr, price)*
Stores available lab tests.

27) **LabReports** *(reportID, appointmentID, reportDate, reportStatus)*
Stores reports for appointments.

28) **ReportItems** *(itemID, reportID, testID, result)*
Stores test results (junction table).

Normalization

1NF (First Normal Form)

Rule: **No repeating groups / multivalued attributes**

BEFORE (Unnormalized Design)

Users(userID | name | phone1 | phone2)

Problem:

- Multiple phone numbers in same row
- Cannot handle unlimited numbers

AFTER (1NF Applied)

Separated multivalued attribute into new table

Users(userID, userName, email)

User_Phones(phoneID, userID, phone)

2NF (Second Normal Form)

Rule: **No partial dependency (depends on full primary key)**

BEFORE (Problem Case)

OrderItems(orderID, productID, productName, price)

Problem:

- productName, price depend only on productID
- Not on full key (orderID + productID)

AFTER (2NF Applied)

Products(productID, proName, price)

OrderItems(orderID, productID, quantity, price)

3NF (Third Normal Form)

Rule: **No transitive dependency**

BEFORE (Problem Case)

Orders(orderID, userID, username, userEmail)

Problem:

- userName, userEmail depend on userID
- Not directly on orderID

AFTER (3NF Applied)

Users(userID, userName, email)

Orders(orderID, userID, totalAmt)

Many-to-Many Resolution

BEFORE

Users ↔ Roles

Orders ↔ Products

Prescriptions ↔ Products

ServiceBookings ↔ Beauticians

LabReports ↔ Tests

Direct many-to-many

AFTER

These are Junction table

user_roles

OrderItems(orderID, productID)

PrescriptionItems

ServiceAssign

reportItems

Queries (LAB PART)/ Procedures/ Triggers

https://drive.google.com/drive/folders/1Fe1j_fwxDPYLVFAhfo4h-KsYNxlzrn1f?usp=sharing

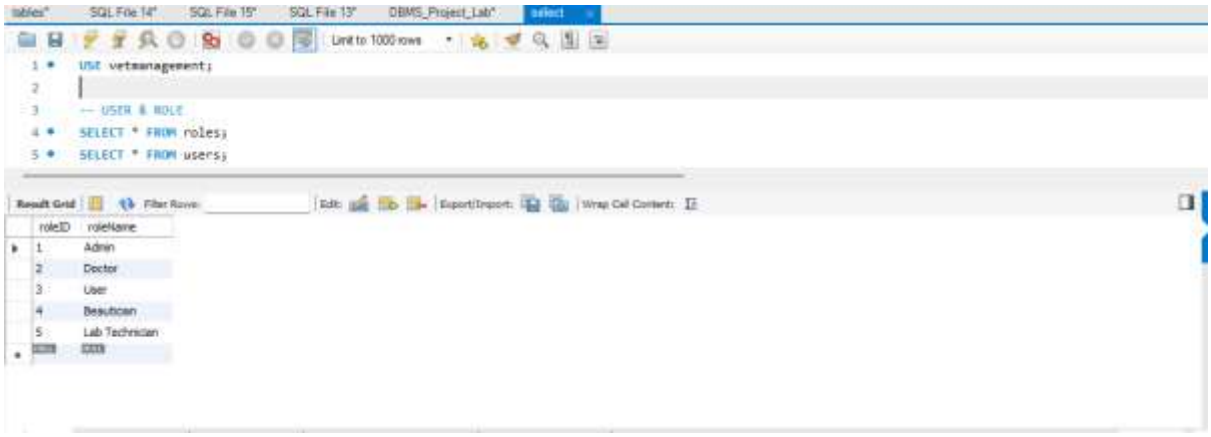
Stored Procedures: 8

1. GetUserOrders
2. AddPet
3. GetTotalSpent
4. CheckStock
5. BookAppointment
6. AddToCart
7. CreateLabReport
8. CheckStockStatus

Triggers: 5

1. trg_reduce_stock: Automatically reduce stock when an order is placed, ensures inventory stays updated, no manual update needed
2. trg_check_stock: Prevent ordering more than available stock
3. trg_log_product_update: Track changes in product (especially price), maintains history, Useful for audit
4. trg_create_inventory: Automatically create inventory record for new product, ensures every product has stock
5. trg_no_negative_stock: Prevent stock from becoming negative, ensures data validity

Snapshots




```
-- lab 4
SET @@SQL_SAFE_UPDATES = 0;
UPDATE products SET price = 550 WHERE productID = 1;
SELECT productID, productName, price FROM products WHERE productID = 1;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

productID	productName	price
1	Dog Food	550.00
NULL	NULL	NULL

tables* SQL File 14* SQL File 15* SQL File 13* DBMS_Project_Lab* select

Limit to 1000 rows

```
SELECT appointmentID,
CASE
    WHEN appointmentStatus = 'Completed' THEN 'Done'
    WHEN appointmentStatus = 'Scheduled' THEN 'Upcoming'
    ELSE 'Other'
END AS status_label
FROM appointments;

SELECT orderID,
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

appointmentID	status_label
1	Done
2	Done

```
-- 7
SELECT u.userName, o.orderID, o.totalAmt FROM users u
INNER JOIN orders o ON u.userID = o.userID;

SELECT p.proName, c.catName FROM products p
JOIN categories c ON p.catID = c.catID;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

userName	orderID	totalAmt
New Name	1	700.00

```
-- 1
DELIMITER //
CREATE PROCEDURE GetUserOrders(IN uid INT)
BEGIN
    SELECT * FROM orders WHERE userID = uid;
END //
DELIMITER ;
CALL GetUserOrders(1);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

orderID	userID	orderDate	totalAmt	orderStatus	addressID
1	1	2026-04-30	700.00	Delivered	1

Result 3

Output

Action Output

#	Time	Action	Message
45	16:35:12	SET @@SQL_SAFE_UPDATES = 0	0 row(s) affected
46	16:35:18	UPDATE inventory SET quantity = quantity - 1 WHERE productID = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0
47	16:35:57	CALL GetUserOrders(1)	Error Code: 1305. PROCEDURE vetmanagement.GetUserOrders does not exist
48	16:36:06	CREATE PROCEDURE GetUserOrders(IN uid INT) BEGIN SELECT * FROM orders WHERE ...	0 row(s) affected
49	16:36:06	CALL GetUserOrders(1)	1 row(s) returned

```
DELETE FROM products WHERE price < 200;

-- DELETE FROM users WHERE userID = 3;
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
56	16:36:55	UPDATE inventory SET quantity = quantity - 1 WHERE productID = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.016 sec
57	16:36:55	UPDATE products SET price = price + (price * 0.10)	5 row(s) affected Rows matched: 5 Changed: 5 Warnings: 0	0.000 sec
58	16:36:55	DELETE FROM cart_items WHERE cartItemID = 2	0 row(s) affected	0.000 sec
59	16:36:55	SELECT * FROM cart_items LIMIT 0, 1000	2 row(s) returned	0.000 sec / 0.000 sec
60	16:36:55	DELETE FROM products WHERE price < 200	Error Code: 1451. Cannot delete or update a parent row: a foreign key constraint fails (`vetmanag...	0.016 sec

We can not delete anything from parent table if any of the columns in it re referenced by any child table column meaning the foreign key referencing a prime key. Therefore we use delete, update cascade.

```sql
-- 13
create table addresses(
                        addressID int auto_increment primary key,
                        userID int,
                        addressLine text,
                        state VARCHAR(50),
                        country VARCHAR(50),
                        city varchar(50),
                        pincode varchar(10),
                        foreign key(userID) references users(userID)
                         ON DELETE CASCADE
                        );
```

Right now quantity is 17 of a product whose productID is 2.

Now let's assume user buys that product in 5 quantity then automatically in inventory it should be reduced and become 12(17-5) due to usage of trigger.

```sql
-- 1
DELIMITER //
CREATE TRIGGER trg_reduce_stock
AFTER INSERT ON orderItems
FOR EACH ROW
BEGIN
    UPDATE inventory
    SET quantity = quantity - NEW.quantity
    WHERE productID = NEW.productID;

    INSERT INTO stockLog(productID, changeType, quantity)
    VALUES (NEW.productID, 'OUT', NEW.quantity);
END //
DELIMITER ;
```

```
    INSERT INTO stocklog(productID, changeType, quantity)
    VALUES (NEW.productID, 'OUT', NEW.quantity);
END //
DELIMITER ;
select * from orders;
select * from products;
select * from inventory;
insert into inventory()value(5,2,17,'2028-12-01');
```

invID	productID	quantity	expiryDate
1	1	37	2027-01-01
2	4	20	2026-12-01
3	5	17	2028-12-01
4	3	15	2028-12-01
5	2	17	2028-12-01

#	Time	Action	Message
63	16:42:38	select * from inventory LIMIT 0, 1000	4 row(s) returned
64	16:43:03	insert into inventory()value(4,2,17,'2028-12-01')	Error Code: 1062. Duplicate entry '4' for key 'inventory.PRIMARY'
65	16:43:14	select * from inventory LIMIT 0, 1000	4 row(s) returned
66	16:43:27	insert into inventory()value(5,2,17,'2028-12-01')	1 row(s) affected
67	16:43:33	select * from inventory LIMIT 0, 1000	5 row(s) returned

A user bought 5 quantity of a product:

```
insert into inventory()value(5,2,17,'2028-12-01');
INSERT INTO orderitems(orderID, productID, quantity, price)
VALUES (1, 2, 5, 500);
select * from orderitems;
```

#	Time	Action	Message	Duration / Fe
68	16:47:11	INSERT INTO orderitems(orderID, productID, quantity, price) VALUES (1, 2, 5, 500)	Error Code: 1062. Duplicate entry '1-2' for key 'orderitems.orderID'	0.047 sec
69	16:47:23	INSERT INTO orderitems(orderID, productID, quantity, price) VALUES (2, 2, 5, 500)	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('vetmanagemse...	0.062 sec
70	16:48:01	select * from orderitems LIMIT 0, 1000	4 row(s) returned	0.000 sec /
71	16:48:44	delete from orderitems where orderItemID=2	1 row(s) affected	0.031 sec
72	16:48:56	select * from orderitems LIMIT 0, 1000	3 row(s) returned	0.000 sec /
73	16:49:05	INSERT INTO orderitems(orderID, productID, quantity, price) VALUES (1, 2, 5, 500)	1 row(s) affected	0.062 sec

Lets check inventory

```
    INSERT INTO stockLog(productID, changeType, quantity)
    VALUES (NEW.productID, 'OUT', NEW.quantity);
END //
DELIMITER ;
select * from orders;
select * from products;
select * from inventory;
```

invID	productID	quantity	expiryDate
1	1	37	2027-01-01
2	4	20	2026-12-01
3	5	17	2028-12-01
4	3	15	2028-12-01
5	2	12	2028-12-01

	Time	Action	Message
69	16:47:23	INSERT INTO orderitems(orderID, productID, quantity, price) VALUES (2, 2, 5, 500)	Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('vetmanagem
70	16:48:01	select * from orderitems LIMIT 0, 1000	4 row(s) returned
71	16:48:44	delete from orderitems where orderItemID=2	1 row(s) affected
72	16:48:56	select * from orderitems LIMIT 0, 1000	3 row(s) returned
73	16:49:05	INSERT INTO orderitems(orderID, productID, quantity, price) VALUES (1, 2, 5, 500)	1 row(s) affected
74	16:50:30	select * from inventory LIMIT 0, 1000	5 row(s) returned

Therefore automatic update done.

```
-- 10
ALTER TABLE users ADD phone VARCHAR(15);
-- WHY: Add new column without recreating table
select * from users;
ALTER TABLE products ADD CONSTRAINT chk_price CHECK (price > 0);
-- WHY: Ensure valid data (no negative price)
ALTER TABLE users MODIFY phone VARCHAR(20);
```

userID	userName	email	password	phone
1	New Name	heerm@gmail.com	123456	NULL
2	Dr. Carol	doctortest@gmail.com	654321	NULL
3	Riya Shah	riyam@gmail.com	234567	NULL
4	Amit Groomer	groomtest@gmail.com	345678	NULL
5	Lab Tech Raj	labtest@gmail.com	456789	NULL

	Time	Action	Message
71	16:48:44	delete from orderitems where orderItemID=2	1 row(s) affected
72	16:48:56	select * from orderitems LIMIT 0, 1000	3 row(s) returned
73	16:49:05	INSERT INTO orderitems(orderID, productID, quantity, price) VALUES (1, 2, 5, 500)	1 row(s) affected
74	16:50:30	select * from inventory LIMIT 0, 1000	5 row(s) returned
75	16:51:40	ALTER TABLE users ADD phone VARCHAR(15)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
76	16:51:40	select * from users LIMIT 0, 1000	5 row(s) returned

```
-- 11
SELECT productName, IFNULL(description, 'No Description') AS desc_value FROM products;
SELECT productName, COALESCE(description, 'Default Description') AS desc_value FROM products;
SELECT p.productName, IFNULL(i.quantity, 0) AS stock
FROM products p
LEFT JOIN inventory i ON p.productID = i.productID;
```

productName	desc_value
Dog Food	Healthy dog food
Cat Toy	Fun toy
Pet Shampoo	Cleaning shampoo
Antibiotic Syrup	Medicine for pets
Comb	comb for pet

Action Output

	Time	Action	Message
72	16:48:56	select * from orderitems LIMIT 0, 1000	3 row(s) returned
73	16:49:05	INSERT INTO orderitems(orderID, productID, quantity, price) VALUES (1, 2, 5, 500)	1 row(s) affected
74	16:50:30	select * from inventory LIMIT 0, 1000	5 row(s) returned
75	16:51:40	ALTER TABLE users ADD phone VARCHAR(15)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
76	16:51:40	select * from users LIMIT 0, 1000	5 row(s) returned
77	16:52:52	SELECT productName, IFNULL(description, 'No Description') AS desc_value FROM products LI...	5 row(s) returned

```
CREATE INDEX idx_product_name ON products(productName);
CREATE INDEX idx_user_email ON users(email);
CREATE INDEX idx_order_user_date ON orders(userID, orderDate);
DROP INDEX idx_product_name ON products;
SHOW INDEXES FROM orders;
```

Result Grid

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
orders	0	PRIMARY	1	orderID	A	0	NULL	NULL		BTREE			YES	NULL
orders	1	addressID	1	addressID	A	0	NULL	NULL	YES	BTREE			YES	NULL
orders	1	idx_order_user_date	1	userID	A	0	NULL	NULL	YES	BTREE			YES	NULL
orders	1	idx_order_user_date	2	orderDate	A	0	NULL	NULL	YES	BTREE			YES	NULL

Result 21

Output

Action Output

	Time	Action	Message
75	16:51:40	ALTER TABLE users ADD phone VARCHAR(15)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
76	16:51:40	select * from users LIMIT 0, 1000	5 row(s) returned
77	16:52:52	SELECT productName, IFNULL(description, 'No Description') AS desc_value FROM products LI...	5 row(s) returned
78	16:53:32	CREATE INDEX idx_product_name ON products(productName)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0
79	16:53:32	CREATE INDEX idx_user_email ON users(email)	Error Code: 1061. Duplicate key name 'idx_user_email'
80	16:53:42	SHOW INDEXES FROM orders	4 row(s) returned

```
drop idx_order_user_date on orders;
SHOW INDEXES FROM orders;

-- 12
-- pk,fk

-- 13
CREATE USER 'test_user'@'localhost'
IDENTIFIED BY '1234';
```

Output

Action Output

	Time	Action	Message	Duration / Fetch
77	16:52:52	SELECT productName, IFNULL(description, 'No Description') AS desc_value FROM products LI...	5 row(s) returned	0.000 sec / 0.000 sec
78	16:53:32	CREATE INDEX idx_product_name ON products(productName)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.125 sec
79	16:53:32	CREATE INDEX idx_user_email ON users(email)	Error Code: 1061. Duplicate key name 'idx_user_email'	0.000 sec
80	16:53:42	SHOW INDEXES FROM orders	4 row(s) returned	0.000 sec / 0.000 sec
81	16:53:40	CREATE INDEX idx_order_user_date ON orders(userID, orderDate)	Error Code: 1061. Duplicate key name 'idx_order_user_date'	0.000 sec
82	16:53:46	drop idx_order_user_date on orders	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to y...	0.000 sec

Grant given

```
-- 13
CREATE USER 'test_user'@'localhost'
IDENTIFIED BY '1234';

GRANT SELECT ON vetmanagement.*
TO 'test_user'@'localhost';

SHOW GRANTS FOR 'test_user'@'localhost';
```

Grants for test_user@localhost
GRANT USAGE ON *.* TO `test_user`@`localh...
GRANT SELECT ON `vetmanagement`.* TO `te...

Result 24

Output

Action Output

#	Time	Action	Message	Duratio
82	16:55:46	drop idx_order_user_date on orders	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to y...	0.000 s
83	16:56:23	CREATE USER 'test_user'@'localhost' IDENTIFIED BY '1234'	Error Code: 1396. Operation CREATE USER failed for 'test_user'@'localhost'	0.078 s
84	16:56:28	GRANT SELECT ON vetmanagement.* TO 'test_user'@'localhost'	0 row(s) affected	0.063 s
85	16:56:33	SHOW GRANTS FOR 'test_user'@'localhost'	2 row(s) returned	0.000 s

Revoked grant

```
GRANT SELECT ON vetmanagement.*
TO 'test_user'@'localhost';

SHOW GRANTS FOR 'test_user'@'localhost';
REVOKE SELECT
ON vetmanagement.*
FROM 'test_user'@'localhost';
```

Grants for test_user@localhost
GRANT USAGE ON *.* TO `test_user`@`localh...

Result 25

Output

Action Output

#	Time	Action	Message	Duration /
81	16:55:40	CREATE INDEX idx_order_user_date ON orders(userID, orderDate)	Error Code: 1061. Duplicate key name 'idx_order_user_date'	0.000 sec
82	16:55:46	drop idx_order_user_date on orders	Error Code: 1064. You have an error in your SQL syntax; check the manual that corresponds to y...	0.000 sec
83	16:56:23	CREATE USER 'test_user'@'localhost' IDENTIFIED BY '1234'	Error Code: 1396. Operation CREATE USER failed for 'test_user'@'localhost'	0.078 sec
84	16:56:28	GRANT SELECT ON vetmanagement.* TO 'test_user'@'localhost'	0 row(s) affected	0.063 sec
85	16:56:33	SHOW GRANTS FOR 'test_user'@'localhost'	2 row(s) returned	0.000 sec
86	16:57:37	REVOKE SELECT ON vetmanagement.* FROM 'test_user'@'localhost'	0 row(s) affected	0.047 sec
87	16:57:41	SHOW GRANTS FOR 'test_user'@'localhost'	1 row(s) returned	0.000 sec

```
-- 14
select * from products;
START TRANSACTION;
UPDATE inventory SET quantity = quantity - 5 WHERE productID = 1;
select * from inventory;
SAVEPOINT sp1;
UPDATE products SET price = 600 WHERE productID = 1;
select * from products;
ROLLBACK TO sp1;
COMMIT;
```

Update in view will also update the main table contents

```
-- 15
-- 1
CREATE VIEW user_view AS SELECT userID, userName, email FROM users;
SELECT * FROM user_view;

-- 2
CREATE VIEW order_summary AS
SELECT u.userName, o.orderID, o.totalAmt
```

userID	userName	email
1	New Name	heerm@gmail.com
2	Dr. Carol	doctortest@gmail.com
3	Riya Shah	riyam@gmail.com
4	Amit Groomer	groomtest@gmail.com
5	Lab Tech Raj	labtest@gmail.com

#	Time	Action	Message	Duration / Fetch
87	16:57:41	SHOW GRANTS FOR 'test_user'@'localhost'	1 row(s) returned	0.000 sec / 0.0
88	16:59:36	CREATE VIEW user_view AS SELECT userID, userName, email FROM users	Error Code: 1050. Table 'user_view' already exists	0.000 sec
89	16:59:43	SELECT * FROM user_view LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.0

Before update main table

```
-- 5
DROP VIEW user_view;

select * from users where userID=1;
-- 6
UPDATE user_view
SET userName = 'New Name'
WHERE userID = 1;
```

userID	userName	email	password	phone
1	New Name	heerm@gmail.com	123456	NULL
NULL	NULL	NULL	NULL	NULL

#	Time	Action	Message
88	16:59:36	CREATE VIEW user_view AS SELECT userID, userName, email FROM users	Error Code: 1050. Table 'user_view' already exists
89	16:59:43	SELECT * FROM user_view LIMIT 0, 1000	5 row(s) returned
90	17:01:18	select * from users where userID=1 LIMIT 0, 1000	1 row(s) returned

Update in view

```
-- 6
UPDATE user_view
SET userName = 'heer mehta'
WHERE userID = 1;
```

#	Time	Action	Message	Duration / Fetch
89	16:59:43	SELECT * FROM user_view LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
90	17:01:18	select * from users where userID=1 LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
91	17:02:10	UPDATE user_view SET userName = 'heer mehta' WHERE userID = 1	1 row(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.078 sec

Also got updated in main table

Future Scope

This project can be further enhanced to evolve into a fully functional and intelligent pet care ecosystem. One major improvement would be the integration of real-time delivery tracking, allowing users to monitor the live location and status of their orders, including medicines, pet products, and other essentials. This would significantly improve transparency and user experience.

Another valuable enhancement is the introduction of a multi-cart system, where users can maintain multiple carts simultaneously for different purposes such as medical needs, daily supplies, or grooming products, providing better organization and flexibility.

Additionally, the system can include a “Notify on Restock” feature, where users can subscribe to out-of-stock products or medicines. Whenever the item is restocked, the system will automatically send a notification to the user. This ensures that users do not miss essential products and improves customer satisfaction.

Further improvements could include integrating a secure online payment gateway, enabling subscription-based services for recurring purchases, and adding smart notifications for appointments, deliveries, and vaccinations. Advanced features like AI-based pet health recommendations and online veterinary consultations can also enhance the system’s functionality.

Adding analytics dashboards for admins and doctors would help in tracking trends, improving decision-making, and optimizing services.

Overall, these enhancements would transform the system into a scalable, user-friendly, and real-world solution that not only manages pet care services but also provides a smarter and more personalized experience for users.

Conclusion

This project represents a complete Pet Care and Veterinary Management System that brings together multiple real-world processes into one seamless workflow. It begins with users registering into the system, where they can manage their pets, book appointments with doctors, and access healthcare services. Once an appointment is scheduled, doctors can generate prescriptions, which are further broken down into individual medicines, ensuring proper tracking and flexibility.

Alongside healthcare, the system also supports an e-commerce module where users can browse products, add items to their cart, and place orders. Each order is processed through structured steps including order items, payments, and deliveries, reflecting real-world shopping experiences. Inventory is automatically managed, with stock updates and logs maintained using triggers to ensure accuracy and automation.

The system further extends into lab management, where tests and reports are generated and linked to appointments, and grooming services, where bookings and staff assignments are handled efficiently. All these modules are interconnected through a well-designed, normalized database structure that minimizes redundancy and ensures consistency.

By integrating advanced DBMS concepts like normalization, joins, triggers, stored procedures, views, indexing, and transactions, the system not only functions efficiently but also mirrors real-life workflows. Overall, this project demonstrates how a structured database can power a scalable, secure, and practical solution for managing pet care services in a unified platform.